Rethinking the Quest for Declarativity

Kathryn Francis

National ICT Australia, Victoria Research Laboratory,
The University of Melbourne, Victoria 3010, Australia
kathryn.francis@nicta.com.au

Abstract. It is generally assumed that in order to free users from the burden of deciding how problems should be solved, it is necessary to provide a more declarative problem specification language. The purpose of this paper is to challenge that assumption, arguing that instead we should treat existing languages more declaratively. Specifically, I argue that the ideal interface to CP would use a (declarative) problem definition written using native code in an existing general purpose programming language.

1 Introduction

The ultimate goal for CP, the holy grail, is that the user simply states the problem and then the computer solves it [3, 4]. This description leaves implicit an important requirement; 'stating' the problem must be easy. Otherwise we are at risk of creating very powerful tools that are never used.

When considering how to make problem specification easy, we need to keep in mind our target users. There is an inherent assumption in many of our tools that the problem will be defined and solved more or less in isolation. This is not realistic for widespread use. In most cases constraint solving will need to be integrated into some sort of larger program. This means both that targeting mathematicians is not especially useful, and that aiming for a problem definition interface simple enough to be used by anyone (e.g. a natural language interface) is unnecessarily ambitious. A more practical choice is to aim for an interface which is intuitive and convenient for application programmers.

The usual suggestion for how problems should be defined involves some sort of abstract, high level problem definition language. Unfortunately this does not make life easy for programmers. It would be much more convenient to allow programmers to define the problem within their own languages, ideally without introducing foreign modelling concepts such as decision variables and constraints. I call this a native interface to CP. The remainder of the paper explains this idea in detail, including illustrative examples in Haskell and Java, before discussing how we might work towards this goal.

2 Native CP interface

Combinatorial problems are difficult to solve because they consist of decisions which cannot be made sequentially. It is not possible to finalise one decision

without considering the options available for the others. This is the difficulty that constraint solving technology should eliminate for programmers.

Rather than insisting that the programmer construct a representation of the problem and send that off to a solver, imagine if they could simply ask some sort of magical oracle to make individual decisions, and use the answers to build a solution. Clearly this would be much easier. Unfortunately it is also impossible. However, it is possible to create an interface which appears to the programmer to be very close to this.

Code which combines individual decisions made by an oracle into a solution defines a pool of candidate solutions. If we also obtain code to evaluate a candidate solution (which should also be straight forward to write), this is enough to define the CSP.

A native interface to CP allows the programmer to define problems in this way and receive solutions exactly as if their own building code had been given a perfect oracle.

Essentially the programmer says:

1. This code builds a candidate solution from individual decisions.
2. This code checks if a candidate solution is feasible.
3. This code calculates the score for a solution. (optional)
4. Build the (best) solution.

Note that the actual computation happens in the last step, and the code does not specify how that step should be achieved. In this sense the interface is declarative even though it may use procedural code.

This final step could be achieved by repeatedly executing the candidate building code with a randomised oracle, each time testing the result for feasibility and score, until one is satisfied that no better solution is likely. This possible implementation allows the programmer to understand what this step means, but obviously it should not be used in practice. Instead we should treat the provided code (1-3) not as code to be executed, but as a declarative specification of the problem to be solved. The library should automatically convert that specification into a standard constraint model, solve it, and then convert the result back into the required form.

3 Show me the code!

In this section examples from two very different programming languages (Haskell and Java) are used to explain in more detail what a native interface to CP might look like and how it would be used. First the interface for each language is defined, and then we examine complete programs solving two example problems. The example problems are taken from the First International Lightning Model and Solve competition, held at CP 2013 [9].

3.1 Haskell

A Haskell version of a native interface to CP is shown in Figure 1. In Haskell we have the advantage that functions are first class. So the three pieces of code provided by the programmer (candidate-building, checking and scoring) can simply be functions which will be passed to the 'find the best solution' function (the interface provides three of these: satisfy, minimise and maximise).

The candidate-building function returns type DeciderState s, where s is the type of a candidate solution. It should make one or more calls to the provided decision functions (chooseBool, chooseInt and chooseOne), combining the results to build a candidate. Since DeciderState is a State this can be done using familiar do notation (as will be shown in the examples). We are using the State monad (rather than having the function simply return type s) to account for the fact that a value of type s will be just one from the pool of candidate solutions. The state within DeciderState is understood to determine which values are returned by chooseBool, chooseInt and chooseOne, and therefore which candidate is produced.

The checking function takes the candidate type s and returns a Bool which is true if the candidate is acceptable, while the scoring function takes an s and returns a value of some orderable type o.

```
module CP where
import Control.Monad.State

type DeciderState = State Int

satisfy :: DeciderState a -> (s -> bool) -> s
minimise :: Ord o => DeciderState s -> (s -> bool) -> (s -> o) -> s
maximise :: Ord o => DeciderState s -> (s -> bool) -> (s -> o) -> s

-- make a yes/no decision
chooseBool :: DeciderState Bool
-- choose a number between the first arg and the second
chooseInt :: Int -> Int -> DeciderState Int
-- choose one item from the given list
chooseOne :: [a] -> DeciderState a
```

Fig. 1: Native Haskell interface to CP

This interface is very simple and easy to understand. There is only one type and very few functions to become familiar with, all of which have a very clear meaning. The type signatures of the main functions provide obvious cues for the programmer about the code they need to write, and that code does not need to use any different syntax or special types (DeciderState is a provided type, but it is really just a State).

3.2 Java

We now consider how the same thing might be achieved in Java. Unfortunately in Java we cannot pass functions as arguments, so instead we define an interface for the programmer to implement.

A class implementing CSP must contain a candidate-building method build, and a checking method check. The build method is passed a Decider object, which provides the same decision functions as in the Haskell version (chooseBool, chooseInt and chooseOne). Instead of returning a candidate it should change the state of the CSP object itself to represent the candidate. The check method will then use this state to determine if the candidate is acceptable. For optimisation problems the programmer must implement the CSPopt interface which extends CSP with another method to calculate a score (Comparable is a standard Java interface for classes having a natural ordering).

This time the 'find the best solution' methods belong to a library class called Solver. The buildMinimal method (for example) can be understood to call the build method of the provided CSP object using a Decider which makes optimal decisions (those producing a state for which calling check will return true and calling score will return the smallest possible value).

```
interface CSP {
  void build(Decider d);
  boolean check();
}

class Decider {
  boolean chooseBool();
  int chooseInt(int min, int max);
  T chooseOne(Set<T> options);
}
```

```
interface CSPopt extends CSP {
  Comparable score();
}

class Solver {
  void buildSatisfactory(CSP problem);
  void buildMinimal(CSPopt problem);
  void buildMaximal(CSPopt problem);
}
```

Fig. 2: Native Java interface to CP

Once again there are very few library classes and methods to become familiar with, and the CSP/CSPopt interface prompts the programmer to write the required code. The decision making code can be more straight forward than the Haskell version because Java has no qualms about methods returning different values each time they are called. However the inability to directly pass functions as arguments makes the semantics of buildSatisfactory/buildMinimal/buildMaximal less clear than that of their Haskell counterparts.

3.3 M-Queens

Our first example problem is M-Queens, which is an optimisation extension of N-Queens. As with N-Queens, the queens placed on the board must not be able to attack each other. The difference is that rather than aiming to place n queens on an n-by-n board, the goal is to place as few as possible while still covering every square on the board (making it impossible to add another queen).

Haskell implementation Figure 3 shows a complete Haskell program solving M-Queens using a native interface to CP. The main function reads the parameter n from standard in, calls mqueens to find an optimal placement of queens for an n-by-n board, and then prints the result to standard out. In the competition from which this example is taken, the required output format for solutions was a list (using the same syntax as Haskell lists) giving the column for the queen in each row, or 0 if no queen is placed in that row. Our program uses this same list representation for solutions to make producing the output convenient (it is sufficient to call the built-in function print as shown).

```
main = do
  n <- readLn
  let chosenCols = mqueens n
  print chosenCols

mqueens :: Int -> [Int]
mqueens n = minimise (choosequeens n) (validqueens n) numqueens      ←

choosequeens :: Int -> DeciderState [Int]                            ←
choosequeens n =
  replicateM n (chooseInt 0 n)                                       ←

validqueens :: Int -> [Int] -> Bool
validqueens n colchoices =
  let allcoords = [(i,j) | i <- [1..n], j <- [1..n]]
      pairs = zip [1..n] colchoices
      chosencoords = filter (\(r,c) -> c>0) pairs
  in all (covered chosencoords) allcoords &&
      noclash chosencoords

covered :: [(Int,Int)] -> (Int,Int) -> Bool
covered chosenpositions pos =
    any (covers pos) chosenpositions

noclash :: [(Int,Int)] -> Bool
noclash coords =
  let pairs = [(c1,c2) | c1 <- coords, c2 <- coords, c1/=c2]
      covers' (a,b) = covers a b
  in not (any covers' pairs)

covers :: (Int,Int) -> (Int,Int) -> Bool
covers (x,y) (i,j) =
  (x == i) || (y == j) || ((x-y) == (i-j)) || ((x+y) == (i+j))

numqueens :: [Int] -> Int
numqueens = length . filter (>0)
```

Fig. 3: Haskell program solving the M-Queens problem.

The mqueens function simply calls minimise passing the three required functions as arguments.

- The candidate-building function choosequeens takes the parameter n and uses replicateM (from State) and chooseInt (from CP) to choose n numbers between 0 and n, returning them in a list. The number at index i represents the column for the queen in row i (with 0 meaning no queen in row i).
- The checking function validqueens takes the parameter n and a list representing a candidate solution and checks that all board positions are covered by the chosen coordinates (using all covered), and no pair of chosen coordinates clash (using noclash). The covers function defines what it means for one coordinate to cover (or clash with) another; the two coordinates must have the same row or column, or the same sum or difference between row and column (as then they are on the same diagonal).
- The scoring function numqueens counts the number of positive numbers in its input list. This corresponds to the number of queens placed on the board.

Note that almost all of this code uses standard types and functions and can be understood independently from the CP module. In fact the CP module is only used on three lines (highlighted with arrows); there is one use of minimize, one use of chooseInt, and the choosequeens function returns a DeciderState. Minimal use of CP-specific code makes this program easy to understand (and to write) for someone familiar with Haskell. Furthermore, the checking functions validqueens, covered, noclash and covers would likely be needed for testing purposes regardless of the solving method.

Java implementation We now consider a Java version of the same program.

In order to demonstrate that we don't have to build the solution in a simple format such as a list of integers, this version instead uses a Map from row number to column number. Only rows in which a queen is placed should be included. Map is a standard Java library class, but it would also be possible (in both the Java and Haskell interface) to build a candidate solution using user-defined classes. For example, another valid representation for the selected queen positions would be a Set of Coordinate objects, each Coordinate having a row and a column.

The majority of the code is shown in Figure 4 (for space reasons print is shown separately in figure 5). The main method retrieves the parameter n, constructs an MQueens object for this size board, then calls buildMinimal to set up a minimal placement before printing the result using the print method of MQueens.

To be accepted as an argument to buildMinimal, the MQueens class needs to implement the CSPopt interface by defining build, check and score.

- The build method iterates through the rows deciding first whether or not a queen should be placed in this row, and then if so which column should be used. The chosen column for each row is recorded in the colForRow map.
- The check method checks that every board coordinate is covered by one of the chosen queen positions (stored in colForRow), and that for every pair of (different) positions in this map, the queens cannot attack each other.

```
class QueensMain {
  public static void main(String[] args) {
    int n = Integer.parseInt(args[0]);
    MQueens mq = new MQueens(n);
    Solver.buildMinimal(mq);                                    ←
    mq.print(System.out);
} }

class MQueens implements CSPopt {                               ←
  int n;
  Map<Integer,Integer> colForRow;
  MQueens(int n) {
    this.n = n;
    colForRow = new HashMap<Integer,Integer>();
  }

  void build(Decider d) {                                       ←
    for(int row = 1; row <= n; row++)
      if(d.chooseBool()) // put a queen in this row?            ←
        colForRow.put(row, d.chooseInt(1, n));                  ←
  }

  boolean check() {
    for(int row = 1; row <= n; row++)
      for(int col = 1; col <= n; col++)
        if(!coveredByChosen(row,col))
          return false; // this position is not covered
    for(int r1 : colForRow.keySet())
      for(int r2 : colForRow.keySet())
        if(r1 != r2 && covers(r1, colForRow.get(r1), r2, colForRow.get(r2)))
          return false; // this pair of queens can attack each other
    return true;
  }
  boolean coveredByChosen(int row, int col) {
    for(int qrow : colForRow.keySet())
      if(covers(qrow,colForRow.get(qrow), row, col))
        return true;
    return false;
  }
  boolean covers(int r1, int c1, int r2, int c2) {
    return (r1==r2 || c1==c2 || (r1−c1 == r2−c2) || (r1+c1==r2+c2));
  }

  Integer score() { return colForRow.size(); }

  // print method excluded
}
```

Fig. 4: Java program solving the M-Queens problem (excluding print method).

– The score method simply returns the size of the map (the number of keys), which gives the number of queens placed on the board.

The print method of MQueens (Figure 5) prints the solution in the required format. Recall that buildMinimal can be understood to call build with an optimal Decider. So when the print method is called (in main immediately after the call to buildMinimal) the colForRow map will contain optimal coordinates.

Once again almost all of the code is independent of the CP library (arrows indicate lines using library methods or classes). Comparing the Haskell and Java solutions, the Java code is unsurprisingly longer (Java is famously verbose), and the Haskell version is more reminiscent of a model written in a dedicated constraint language (with its use of not, any and all), but they are actually very similar. Which version is easier to understand would almost certainly depend on the familiarity of the reader with the two different languages. For this reason I believe we should create native interfaces for a variety of languages rather than attempting to choose a single language which is most suited to the task.

```
void print(PrintStream ps) {
  ps.print("[");
  for(int row = 1; row <=n; row++) {
    if(row > 1)
      ps.print(",");
    if(colForRow.containsKey(row))
      ps.print(colForRow.get(row));
    else ps.print(0);
  }
  ps.println("]");
}
```

Fig. 5: Print method extracted from MQueens class in Figure 4.

3.4 Doubleclock

Our second example is based on the solitary card game double clock patience. Assuming a deck with $2k$ suits and n different card numbers, the rules of the game are as follows. The cards are dealt into $2n$ piles (of k cards), and then the top-most card in the last pile is revealed and moved to the initially empty 'discard' pile. The player then makes a sequence of moves, each time discarding another card, until all piles are empty. In each move the number (ignoring the suit) of the last discard determines which two piles the next card might be taken from. If the number is j then the player can choose pile $2j-1$ or pile $2j$. If one of these piles is empty then the other pile must be chosen. If both are empty then the next card is taken from the last non-empty pile, incurring a restart penalty.

Our objective is to find a sequence of moves minimising the number of restarts. The input data consists of the two size parameters n and k followed by a permutation of $1..2nk$ giving the initial layout of the cards. The first k integers

give the first pile from bottom to top, the next k integers the second pile, and so on, where integer j represents a card with number $(j-1) \mod n+1$. The required output is another list of integers giving the discard order.

The Haskell and Java implementations are largely the same. To make input and output straight forward we use the given integer representation for cards. Both versions essentially simulate the game, counting the number of restarts and recording the discards. At each move only allowable choices are made, so all candidate solutions will be valid.

The Haskell program (Figure 8) passes the input data straight through to the doubleclock function, where it is converted into the two parameters n and k and a list of list of Int for the piles. Then minimise is used to find an optimal solution. The candidate-building function is play, with an initial state constructed by discarding the top card from the last pile $(2n)$. A candidate solution produced by play is a pair composed of the discard list and number of restarts. We want to minimise the number of restarts, so the scoring function is snd (which returns the second element from a pair). The checking function is simply true.

The play function simulates the game. If all piles are empty then the game is finished and the current discard pile and number of restarts are returned. Otherwise we first find the available choices for the next discard, and then either perform a restart (if neither pile contains cards) or use chooseOne to decide on a pile and remove a card from there, before continuing with the game.

In the Java program (Figures 6 and 7) the game state is represented using a list of stacks (each stack is a pile). The state is initialised in the DoubleClock constructor using the input data passed in from main. The build method makes the compulsory first move and then continues making allowed moves until the game is finished (all piles are empty). When choosing a pile, if only one option is available then that pile is selected. If no options are available then the number of restarts is incremented and the last non-empty pile is selected. Otherwise chooseBool is used to decide which pile to select. As with the Haskell version, the check, score and print methods are trivial.

Once again both versions contain very little CP-specific code (all affected lines are highlighted with arrows). A competent programmer should find it straight forward to produce similar programs. The Java implementation feels more natural in this case; as a procedural language it is well suited to simulating the game playing process. This demonstrates that the choice of specification language may depend not only on the programmer's experience but also on the nature of the problem, highlighting the need to support multiple languages.

```
public class ClockMain {
  public static void main(String[] args) {
    DoubleClock dc = new DoubleClock(args);
    Solver.buildMinimal(dc);                    ←
    dc.print(System.out);
} }
```

Fig. 6: Main method for Java doubleclock program.

```
public class DoubleClock implements CSPopt {                              ←
  List<Stack<Integer>> piles = new ArrayList<Stack<Integer>>();
  Stack<Integer> discardPile = new Stack<Integer>();
  int restarts = 0;
  int n;
  DoubleClock(String[] args) {
    n = Integer.parseInt(args[0]);
    int k = Integer.parseInt(args[1]);
    Stack<Integer> pile = new Stack<Integer>();
    piles.add(pile);
    for(int i = 2; i < args.length; i++) {
      if(pile.size() == k) {
        pile = new Stack<Integer>();
        piles.add(pile);
      }
      pile.push(Integer.parseInt(args[i]));
  } }

  public void build(Decider d) {                                           ←
    discardPile.push(piles.get(piles.size()−1).pop()); // compulsory first move
    while(!gameFinished()) {
      Stack<Integer> chosenpile = choosePile(d, discardPile.peek());       ←
      discardPile.push(chosenpile.pop());
  } }

  Stack<Integer> choosePile(Decider d, int lastcard) {                     ←
    int cardnum = ((lastcard−1) % n) + 1;
    Stack<Integer> pile1 = piles.get(2∗cardnum−1);
    Stack<Integer> pile2 = piles.get(2∗cardnum−2); // 0−based index
    if(pile1.isEmpty() && !pile2.isEmpty()) return pile2;
    if(pile2.isEmpty() && !pile1.isEmpty()) return pile1;
    if(pile1.isEmpty() && pile2.isEmpty()) {
      restarts++;
      for(int i = piles.size()−1; i >=0; i−−) // find last nonempty pile
        if(!piles.get(i).isEmpty()) return piles.get(i);
    }
    return d.chooseBool() ? pile1 : pile2;                                 ←
  }

  boolean gameFinished() {
    for(Stack<Integer> p : piles)
      if(!p.isEmpty()) return false;
    return true;
  }
  public boolean check() { return true; }
  public Integer score() { return restarts; }
  void print(PrintStream ps) { ps.print(discardPile); }
}
```

Fig. 7: Java CSPopt implementation for the doubleclock problem.

```
type Card = Int
data Gamestate = Gamestate Int [[Card]] [Card] Int

main = do
  i <- readLn
  inputdata <- readFile ("doubleclock_"++i++".txt")
  let (discards, restarts) = doubleclock inputdata
  print discards

doubleclock :: String -> ([Card], Int)
doubleclock inputdata =
  let [n:k:cards] = map read (words inputdata)
      piles = chunksOf k cards
      initialstate = makemove (2*n) (Gamestate n piles [] 0)
  in minimise (play initialstate) (const True) snd                    ←

play :: Gamestate -> DeciderState ([Card], Int)                       ←
play gs@(Gamestate n piles discardpile restarts) =
    if all null piles
    then return (discardpile, restarts)
    else let j = cardnumber (head discardpile) n
             pileoptions = filter (notEmpty piles) [2*j, 2*j-1]
         in
             if null pileoptions
             then play (restart gs)
             else do
                 chosenPile <- chooseOne pileoptions                  ←
                 play (makemove chosenPile gs)

cardnumber :: Card -> Int -> Int
cardnumber card n = ((card - 1) `mod` n) + 1

notEmpty :: [[a]] -> Int -> Bool
notEmpty piles i = not (null (piles !! i))

makemove :: Int -> Gamestate -> Gamestate
makemove pileindex gs@(Gamestate n piles discard restarts) =
  let (earlypiles, chosenpile:latepiles) = splitAt pileindex piles
      card = head chosenpile
      piles' = earlypiles ++ (tail chosenpile:latepiles)
  in Gamestate n piles' (card:discard) restarts

restart :: Gamestate -> Gamestate
restart (Gamestate n piles discard restarts) =
  let remainingpiles = filter (notEmpty piles) [1..(length piles)]
  in makemove (last remainingpiles) (Gamestate n piles discard (restarts+1))
```

Fig. 8: Haskell program solving the doubleclock problem.

4 Advantages of a native interface

If our target users are programmers then there are significant advantages to allowing problems to be defined using existing general purpose programming languages. Programmers are already familiar with and practiced at using these languages, mature editors/IDEs exist, and integration with a wider program (which is essential for widespread use of CP) is much easier.

For many programming languages there already exist one or more CP libraries designed to facilitate the integration of constraint solving into a wider program. The following is a list of advantages of a native interface over one of these libraries. Some of these advantages also apply when comparing to dedicated problem definition languages.

4.1 No CP-specific modelling

While using a standard CP library is certainly more convenient than using a separate language, the programmer still has to learn CP-specific modelling skills. With a native interface that is not the case. The programmer does not have to learn how to write a CP model, coming to grips with decision variables and constraints. Unlike using a standard library it is not necessary to understand anything about how constraint solvers work.

Obviously the programmer still needs to choose a representation for solutions. This is modelling, but since the choice is based on the requirements of the application rather than the requirements of the solver, it is no different from the modelling already required to write the rest of the application.

4.2 Less documentation

CP libraries typically require writing code which makes heavy use of library types representing decision variables and constraints. For example, in the n-queens sample program provided with Choco [7], 16 of the 26 lines of code (excluding imports, annotations, comments, bracket-only lines and whitespace) use a library class or method. This is inconvenient for programmers who have to look up how to use each library-provided type or function. Apart from being a hassle it can also be very daunting. The Gecode [8] tutorial is more than 500 pages long. Even Numberjack [6], which is aimed more at beginners, has 36 classes with almost 70 documented members. In contrast, a native interface introduces a very small number of types and methods and should require very little documentation. This means programmers can quickly read the entire specification and not wonder what else is there that they are not aware of.

4.3 Readable code

The requirement to use special types representing decision variables also often necessitates special syntax or awkward code which obfuscates the problem definition, especially when dealing with mathematical expressions. While this can

sometimes be avoided using operator overloading, with a native interface there is no need to introduce special types in the first place, so the code is readable even in languages such as Java which do not allow operator overloading.

4.4 Reusable code

When using a native interface, code forming part of the problem definition can be re-used, for example to evaluate current practice or user-provided solutions. The data structures used to represent the input data and solution can also be re-used. For example, there is no need to invent an integer representation for cards if the rest of the card playing application uses a structured type. The same structured type can be used directly in the candidate-building, checking and scoring code. Any required conversion between solver and host language types is performed automatically.

4.5 Easier debugging and testing

Native interfaces permit natural, straight forward debugging. Incorrect solver results indicate a bug in the building or checking code. An invalid solution gives a failing test case for the checking code (it returns true for the given candidate when it should return false), while a missing solution indicates either that the building code cannot build that solution, or that the checking code erroneously returns false. Once one of these problems has been discovered, it can be corrected with no further reference to a constraint solver, using standard debugging techniques and tools (e.g. interactive debuggers or logging). Narrowing down which constraint is erroneous in a standard model is often much harder (in part because fewer tools are available).

Similarly, the code defining the constraint problem can be tested in the same manner as the rest of the program, without using a constraint solver. Individual methods/classes/functions used in the definition may have their own unit tests. Even the decision-making code can be tested independently, using a Decider/DeciderState implementation making random or predetermined decisions.

4.6 IDEs and compile-time checking

A native interface allows advanced features of IDEs such as text-prediction, automatic re-factoring, and live compilation to be directly applicable to the code defining the model. Standard static checking performed by the compiler can help prevent errors in the problem definition, with no need for error-prone string-based references to variables.

5 Implementing a native interface

It might seem as though the described interfaces are impossible to implement. This is certainly not the case, in fact in previous work [1] I have implemented

a proof-of-concept Java interface similar to the one described here. In order to understand how this is possible we need to come back to the idea of treating code written in existing languages more declaratively.

Our basic requirement is for the three pieces of code provided by the programmer (candidate-building, checking and scoring) to be translated into a constraint model. This can be achieved using symbolic execution techniques. All uncertainty in the computation stems from the decision making calls. Each of these requires the creation of one or more decision variables to represent the returned value. The rest of the code can be translated into constraints linking these values to the computed checking result and score. Note that inputs already provided to functions passed to minimise (for example), or the state of the program when buildMinimal is called, can be considered parameters as they will be known at solving time (when minimise/buildMinimal is actually called).

Given a model produced in this fashion, a standard constraint solver can be used to find satisfactory/optimal results for the decision-making calls. These can then be used to execute the candidate-building code, producing the result expected by the programmer. The practical details of when the translation should be performed and exactly how the correct final state should be produced will depend on the language. For an example of a possible approach for Java see [1].

While a basic translation is reasonably straight forward, the resulting models are likely to be very inefficient. The following list outlines some ideas for how a more effective translation might be achieved (also see [2]).

- Functions/methods for commonly used types like lists and sets can be translated directly as atomic operations.
- Specialised translations can be used for common code patterns (e.g. computing a sum), possibly building on pattern-detection algorithms already used for editor suggestions.
- Compiler transformations designed to produce more efficient code might be adapted to produce more effective models. These two goals are related as they both benefit from simplicity.
- The structure of the code could be used to aid the automatic detection of global constraints. Looping, recursion, and the use of higher-order functions (e.g. all) suggest that a global constraint may be relevant.
- Other structure information present in the code may also be useful. For example, nested decision making as in the Java M-Queens program (Figure 4) may benefit from a particular search strategy or other solving technique.
- Given a correct model, we can apply general model reformulation techniques.

Obviously a translation able to compete with models produced by a human expert is a very ambitious goal. I am certainly not suggesting we abandon work on dedicated modelling languages or CP libraries. However, as a starting point we can aim for a translation sufficient for small/easy problems, or we can make concessions in our interface to allow greater control over the resulting model (e.g the plugging in of specialised constraints and search strategy). Also, while chasing our ultimate goal we will almost certainly encounter ideas which could be transferred back to the mainstream approach. Two examples are given below.

5.1 Complex decisions

The decision functions included in the examples above are sufficient to represent any problem, but in practice the interface should include more complex decisions such as choosing a permutation of a list, or a partition of a set. One might also provide more specific decisions like scheduling a set of tasks so that none overlap. The benefit is that appropriate (global) constraints can be added automatically.

Global constraints are often used to tie back together complex decisions which have been decomposed. It would be preferable to allow the direct specification of the original decision, enabling automatic symmetry breaking and straight forward experimentation with different decompositions. This is the premise behind Essence [5], which allows decision variables to represent arbitrarily complex combinatorial objects. However, it should be possible to encapsulate more constraints within a complex decision, beyond the type of the combinatorial object.

It would also be interesting to see how this idea could be incorporated in a normal CP library. Ideally the actual decision variables would be hidden behind an abstract type for the complex decision, so that different decompositions could be compared without changing the rest of the code.

5.2 Separation of decisions and consequences

A native interface provides a clear distinction between core decisions (calls to the library-provided decision functions) and their consequences (values computed from the results). This information could be used to guide the search, or avoid useless propagation. It also prevents modelling errors where the consequences are not actually functionally defined by the decisions. It might be possible to transfer some of these benefits to mainstream modelling if we could introduce a distinction between core decisions and consequences.

6 Conclusion

The results of the 2013 International Lightning Model and Solve competition served as a reminder that our tools are too hard to use. The winners solved the problems by hand, suggesting that creating an effective model was more time consuming (or daunting) than solving the problems. In order for the holy grail (that users simply define the problem and then the computer solves it) to solve our usability problems, we need to ensure that it is not still easier to solve the problem by hand than to define it in a way the computer understands.

In this paper I argue that we should be aiming for a realisation of the holy grail where constraint problems are defined natively inside general purpose programming languages. The benefit of this approach is greatly enhanced ease of use for programmers, who represent ideal target users due to their ability to embed constraint solving in wider applications.

Acknowledgments NICTA is funded by the Australian Government through the Department of Communications and the Australian Research Council through the ICT Centre of Excellence Program.

References

1. K. Francis, S. Brand, and P.J. Stuckey. Optimization modelling for software developers. In M. Milano, editor, *Proceedings of the 18th International Conference on Principles and Practice of Constraint Programming*, number 7514 in LNCS, pages 274–289. Springer, 2012.
2. K. Francis, J. Navas, and P.J. Stuckey. Modelling destructive assignments. In C. Schulte, editor, *Proceedings of the 19th International Conference on Principles and Practice of Constraint Programming*, volume 8124 of *LNCS*, pages 315–330. Springer, 2013.
3. E. C. Freuder. In pursuit of the holy grail. *Constraints*, 2(1):57–61, 1997.
4. E. C. Freuder. Holy grail redux. *Constraint Programming Letters*, 1(1):3–5, 2007.
5. A.M. Frisch, W. Harvey, C. Jefferson, B. Martnez-Hernndez, and I. Miguel. Essence: A constraint language for specifying combinatorial problems. *Constraints*, 13(3):268–306, 2008.
6. E. Hebrard, E. O'Mahony, and B. O'Sullivan. Constraint programming and combinatorial optimisation in Numberjack. *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR)*, pages 181–185, 2010.
7. N. Jussien, G. Rochart, and X. Lorca. The CHOCO constraint programming solver. In *CPAIOR08 Workshop on OpenSource Software for Integer and Constraint Programming OSSICP08*, 2008.
8. C. Schulte, M. Lagerkvist, and G. Tack. GECODE – an open, free, efficient constraint solving toolkit. http://www.gecode.org/.
9. P. J. Stuckey, H. Kjellerstrand, and organisers. First international lightning model and solve competition. http://cp2013.a4cp.org/program/competition, 2013.